

Problem 1

Consider the following algorithm SILLYSORT and its subroutine SMERGE. Aptly named, SILLYSORT is a MergeSort-like algorithm that sorts the given array $R[1..n]$ of n integers. For all parts, assume that n is a power of two and R contains distinct integers (no duplicates).

```

SILLYSORT( $R[1..n]$ ):
  if  $n > 1$ :
    SILLYSORT( $R[1..n/2]$ )
    SILLYSORT( $R[n/2 + 1..n]$ )
    SMERGE( $R[1..n]$ )

```

```

SMERGE( $R[1..n]$ ):
  if  $n = 2$ :
    if  $R[1] > R[2]$ :
      swap  $R[1] \longleftrightarrow R[2]$ 
  else:
    for  $i \leftarrow 1$  to  $n/4$ :
      swap  $R[i + n/4] \longleftrightarrow R[i + n/2]$     «swap middle fourths»
    SMERGE( $R[1..n/2]$ )    «leftmost half»
    SMERGE( $R[n/2 + 1..n]$ )    «rightmost half»
    SMERGE( $R[n/4 + 1..3n/4]$ )    «center half»

```

Note: The notation $R[i..j]$ is a pointer to the subarray of an array R from indices i to j (inclusive). Any operations on a subarray are in-place and thus occur on the original array that contains it. (This is the same as for the description of MERGESORT and QUICKSORT from our textbook/lecture.)

- (a) Use induction to prove that $\text{Smerge}(R[1..n])$ sorts the given array R in-place, assuming that $R[1..n/2]$ and $R[n/2 + 1..n]$ are sorted. [Hint: It may be useful to first follow the smallest $n/4$ and largest $n/4$ elements in R before/between/after the initial for-loop and recursive calls.]

Solution:

Let n be an arbitrary power of 2 and Smerge correctly sorts array $R[1..k]$ for all $k < n$. We want to show that $\text{Smerge}(R[1..n])$ correctly sorts the array.

Base case: When $n = 1$, the array is trivially sorted. If $n = 2$, then by construction, $R[1..2]$ is sorted.

Inductive Case: Consider $n > 3$ and a power of 2. For simplicity, partition $R[1..n]$ into 4 contiguous sections, each with $n/4$ elements. We denote index these sections with $i \in \{1, 2, 3, 4\}$ respectively. Given that $R[1..n/2]$ and $R[n/2 + 1..n]$ are sorted, before calling $\text{Smerge}(R[1..n])$, we know that the smallest $n/4$ elements of $R[1..n/2]$ and $R[n/2 + 1..n]$ live in sections 1 and 3 of $R[1..n]$ respectively. And largest $\frac{n}{4}$ elements of each lists live in sections 2 and 4 respectively. Thus, the for loop call within $\text{Smerge}(\cdot)$ will modifies $R[1..n]$ with the smallest $n/4$ elements of $R[1..n/2]$ and $R[n/2 + 1..n]$ in sections 1 and 2 respectively. By our inductive hypothesis, $\text{Smerge}(R[1..\frac{n}{2}])$ implies that the smallest $\frac{n}{4}$ elements live in section 1 and is sorted, and largest $\frac{n}{4}$ live in section 4 and is also sorted. Hence, calling $\text{Smerge}(R[n/4 + 1..3n/4])$ will sort the middle $[n/4 + 1..3n/4]$ section, and $R[1..n]$ will be sorted upon termination.

- (b) State a recurrence that describes the runtime of Smerge , then solve it using the recursion tree method.

Solution:

Swapping the middle fourths performs $\frac{n}{4}$ operations. Therefore, our recurrence is:

$$T(n) = 3T\left(\frac{n}{2}\right) + \frac{n}{4}.$$

For each recursion, we first do swap, which performs $n/4$ operations. Then we are calling 3 nodes with lists of $n/2$ length. For each node, the swap performs length/4 operations. Therefore, each layer k takes $\frac{3^{k-1} * n}{2^{k+1}}$ time. Thus, the total time should be: $n \sum_{i=1}^{\log_2(n)} \frac{3^{i-1}}{2^{i+1}}$. It follows that our asymptotic runtime is $O(n^{\log_2 3})$.

- (c) State a recurrence that describes the runtime of SillySort, then solve it using the recursion tree method.

Solution:

Using the result from before, our recurrence relation is

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n^{\log_2(3)}).$$

We have $\log n$ layers. Hence, our runtime is

$$O(n^{\log_2(3)} \log n).$$

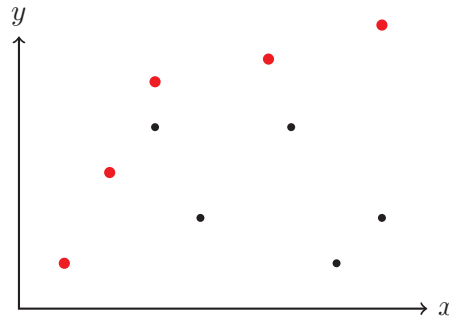
Additional ungraded exercise—do not submit: Prove that $\text{SILLYSORT}(R[1..n])$ indeed sorts R using induction, assuming the correctness of Smerge from part (a).

Solution:

TODO: Complete. For readability, it is preferred not to submit your final solutions with the entire problem statement included.

Problem 2

Let $S = \{p_i = (x_i, y_i) : 1 \leq i \leq n\}$ be a set of n points on a 2D plane. We say that a point p_i is *northwest* (NW) of p_j if $x_i \leq x_j$ and $y_i \geq y_j$, treating the $(-x)$ -direction (resp., $(+y)$ -direction) as west (resp., north). A point p_i is a *northwesterly point* of S if there are no other points in S NW of p_i . In the example below, the larger red points are the northwesterly points of S .



Describe an algorithm that returns the northwesterly points of S in $O(n \log n)$ time. Prove the correctness of your algorithm using induction, then state and justify its runtime using the recursion tree method. [Hint: Sort then divide S into two sets based on the x -coordinates of points, and “conquer.” What is the appropriate merge/combine step to obtain the overall solution?]

```
Def NW(P[(x1, y1), ..., (xn, yn)]):
    SortX(P[(x1, y1), ..., (xn, yn)])
    FindNW(P[(x1, y1), ..., (xn, yn)])
```

```
Def FindNW(P[(x1, y1), ..., (xn, yn)])
    if n = 2:
        if P(y1) > P(y2):
            exclude P(x2, y2)
    L = FindNW(P[(x1, y1), ..., (xn/2, yn/2)])
    R = FindNW(P[(xn/2+1, yn/2+1), ..., (xn, yn)])
    for i ← 1 to len(R):
        if L(ylen(L)) > R(yi):
            exclude R((xi, yi))
    return concat(L, R)
```

Solution:

Let n be arbitrary and `findNW` correctly returns all northwesterly (nw) points within $R[1..k]$ for all $k < n$. We want to show that `findNW`($R[1..n]$) also holds.

Base case: When $n = 1$, the array trivially contains all nw points. If $n = 2$, then by construction, `findnw` returns all nw points.

Inductive Case: By inductive hypothesis, L contains all nw points in $P[(x_1, y_1), \dots, (x_{n/2}, y_{n/2})]$, and R for $P[(x_{n/2}, y_{n/2}), \dots, (x_n, y_n)]$. By definition of nw points and because our list is sorted according to x coordinates, it suffices to check the right most point of L against all points of R (recall that for $\text{len}(L) > 1$, if $y_{\text{len}(L)} > y_{r_i}$ where $x_{\text{len}(L)} \leq x_{r_i}$ $i \in 1, \dots, \text{len}(R)$, (x_{r_i}, y_{r_i}) is not nw). Therefore, we obtain all nw points upon termination.

Runtime: For each recursion, we first call 2 nodes with lists of $n/2$ length. Then we do for loop that takes $i \leq n/2$ operations. Thus, each node takes less than $n/2$ operations. Therefore, each layer i takes less than $\frac{2^{(i-1)} * i}{2^{(i-1)}} = i - 1$ time. Therefore, the total time should be: $\sum_{i=1}^{\log_2(n)} (i - 1)$. It follows that the asymptotic runtime is $O(n \log n)$.

Problem 3

An array $A[1..n]$ of n distinct integers (no duplicates) is considered *twirled* if it consists of two non-empty contiguous sequences, L and R , that are sorted in increasing order and all elements in L are greater than those in R . In other words, if R instead appeared left of L , the entire resulting sequence would be sorted in increasing order.

For example, in the following twirled array, the left subsequence is underlined and has length 6, the right subsequence is overlined and has length 4, and indeed all elements in the left subsequence are greater than those in the right subsequence:

$[\underline{12, 14, 19, 23, 25, 27}, \overline{2, 3, 5, 7}]$

Describe an $O(\log n)$ -time algorithm to return the length of the left subsequence of a given twirled array A . Prove its correctness using induction, then state and justify its runtime using the recursion tree method.

[Hint: Consider a variant of binary search. Given an element x of A , what information do you need to decide which of the left and right subsequences that x belongs to?]

Solution:

```

Def LenLSearch(A[1..n], x=1, y=n):
  if y-x = 1:
    return x
  m = ⌊ $\frac{x+y}{2}$ ⌋
  if A[x] < A[m]:
    LenLSearch(A[1...n], m, y)
  else:
    LenLSearch(A[1..n], x, m)

```

Correctness proof:

Consider arbitrary n and assume $LenLSearch(A[1..n], x, y)$ correctly returns the length of left subsequence for any integer inputs x and y such that $1 \leq y - x < n - 1$, $1 \leq x < y \leq n$, and A is a twirled array. We want to show that $LenLSearch(A[1..n], x = 1, y = n)$ returns the correct length of the left subsequence for $y - x = n - 1$.

Base case: See that if $n = 1$, i.e. $y - x = 1$ then we are considering array $A[x, y]$ of size 2. By definition of twirled array, x is the end of the left subsequence and such index is also the length of the left subsequence.

Inductive step: Suppose $n \geq 3$. We first note that $A[x]$ will always be an element in L . If $A[x] < A[m]$, then $A[m]$ is contained in left array, since all elements of left array are greater than those of right array, and $x = n$. x remains unchanged in any other cases.

If $A[x] < A[m]$, then we know that by the definition of twirled array, the last element of the left array should be in $A[m..n]$. And we have

$$y - m = y - \left\lfloor \frac{x+y}{2} \right\rfloor \leq y - \frac{x+y-1}{2} = \frac{y-x+1}{2} \leq \frac{n-1+1}{2} < n-1$$

Therefore, by inductive hypothesis, $LenLSearch(A[1..n], x = m, y = n)$ correctly returns $\text{len}(L) = x$.

If $A[x] > A[m]$ then $A[m]$ is contained in right array, and the last element of left array should be in $A[x..m]$. Since $x \leq m < y < n$, we have:

$$m - x = \left\lfloor \frac{x+y}{2} \right\rfloor - x \leq \frac{x+y}{2} - x = \frac{y-x}{2} < \frac{n-1}{2} < n-1$$

by inductive hypothesis, $LenLSearch(A[1..n], x, m)$ returns correctly returns $\text{len}(L) = x$.

Runtime: In worst case, the length is found at base case. In every recursion step, the middle value is compared with the first value, which takes constant time. Then the list is divided by half and the last element of left sequence lies in one of them. Thus, the recursion tree will have $\log_2 n$ layers, with each layer taking constant time. The runtime will be at most $O(\log n)$.

Problem 4

In the following, all arrays/sequences are 0-indexed (as in Python). A *flippable list* is a data structure f that stores a finite sequence S_f of distinct integers and supports the following two operations:

- $\text{PEEK}(i)$: Return the object at index i in S_f , where 0 is the first index.
- $\text{FLIP}(i, j)$: Reverse (“flip”) the subsequence $S_f[i..j-1]$ if $i < j-1$ and do nothing otherwise. (NOTE: Notice the $j-1$ in the definition. This is to mimic Python’s syntax: `x[i:j]` returns the sublist of list `x` from `i` to `j-1`.)

Example: If $S_f = [4, 5, 2, 1, 3]$, $\text{PEEK}(1)$ returns 5 and $\text{FLIP}(1, 4)$ modifies S_f so that it becomes $[4, \underline{1}, \underline{2}, \underline{5}, 3]$, where the underlined symbols are those in the affected subsequence.

For this problem, you will complete an implementation of a method similar to the `PARTITION` subroutine for `QUICKSORT` in Python 3. Download the class file `flippable_list.py` and your template file `flartition.py` on Canvas in the Files section. The file `flippable_list.py` includes the `FlippableList` class, which supports the `PEEK` and `FLIP` operations. For any Python list S , a `FlippableList` object representing S is constructed and returned by calling `FLIPPABLELIST(S)`. **You may not directly access the `_lst` attribute of the `FlippableList` objects in your submission, otherwise you will receive a grade of 0.** However, you may find checking it to be useful while testing your code. See also the helper functions provided in the file.

`flartition.py` is the file in which you will complete the following task and cite any/all collaboration before uploading it to Gradescope under “HW1 Problem 4 (Code)”. This problem has no written component.

Let f be a flippable list, let i, j be indices of S_f , and let p be an integer (the “pivot” value). Design and implement a recursive method `FLARTITION( $f, i, j, p$ )` that modifies the subsequence $S_f[i..j-1]$, using only `PEEK` and `FLIP` operations on f , and returns an index k where:

1. all elements in $S_f[i..k-1]$ are all *less than* p , and
2. all elements in $S_f[k..j]$ are all *at least* p .

In other words, the returned index k is the first index of an element in the modified subsequence that is at least p ; if all elements in the subsequence are less than p , then $k = j$.¹

Example: For flippable list f with sequence $S_f = [4, 9, 330, 1, 2, 5, 8, 16]$, a valid value of S_f after `FLARTITION( $f, 2, 7, 6$ )` is $[4, 9, \underline{2}, \underline{1}, \underline{5}, \underline{330}, 8, 16]$, where the underlined elements are those in the affected subsequence. The return value of the call is 5, which is the index of the first element (330) at least the pivot value of 6. The orderings of the elements less than/at least the pivot value 6 are arbitrary.

[Hint: Split the problem into two halves, conquer, and then finish. Note that the median index of a subsequence $S_f[i..j-1]$ can be computed as $(i+j)/2$, rounded down.]

¹Note that this method differs from the `PARTITION` subroutine from `QUICKSORT` in the notes/lecture. In particular, the parameter p need not be in $S_f[i..j-1]$, and elements equal to p may end up anywhere after the elements smaller than p , not necessarily before those greater than p .